



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems
 Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
 Example Logic: IF `food_here` THEN `suck`; IF `wall_ahead` THEN `turn_left`; ELSE `move_forward`
3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
 Example Logic: IF `wall_ahead` AND `left_is_visited` THEN `turn_right`

5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent must store an action for every possible percept sequence. In complex environments such as chess, the number of possible sequences is extremely large. As the agent's lifetime increases, its percept history grows, causing a combinatorial explosion. Therefore, storing and searching the complete table becomes impossible.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The condition-action rules in the `SimpleReflexAgent` are:

```
if percept['food_here']:
    return 'Suck'
```

```
if percept['wall_ahead']:
    return 'TurnLeft'
```

```
return 'MoveForward'
```

These follow the "IF condition, THEN action" structure. The agent sucks when food is present, turns left when a wall is ahead, and otherwise moves forward. It uses only the current percept and does not store history.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The environment is partially observable because the agent receives only local information and cannot see the complete map or its exact position. It also has no memory of previous percepts or visited locations. Therefore, when it encounters the same local situation again, it chooses the same action and may become trapped in an infinite loop.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The `ModelBasedAgent` stores its internal state using the following:

```
self.percept_history
self.last_action
self.repeat_count
```

The sensor model uses local percepts such as `wall_ahead` and `food_here` to describe the agent's current surroundings. The transition model uses the previous action and percept history to estimate whether the agent's actions are leading to a repeated situation. The agent updates this memory before choosing its next action. When it detects a loop, it turns right and attempts an alternative path.

